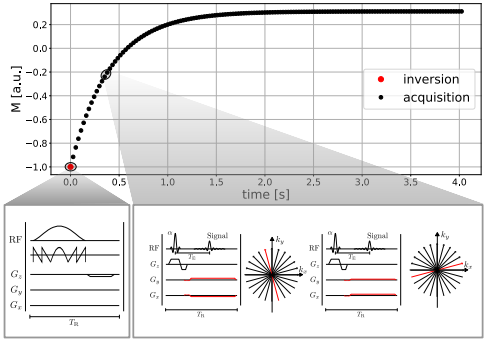
BART Sequence Framework

Daniel Mackner
Institute of Biomedical Imaging
Graz University of Technology

Vendor-Agnostic Pulse Sequence Programming with Pulseq II

February 24th, 2026

Quantitative (Model-based) MRI Techniques

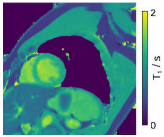


+ BART

$$S(\boldsymbol{x_p}(r), t) = M_{ss}(r) - (M_{ss}(r) + M_0(r))\, e^{-t/T_1^*(r)}$$

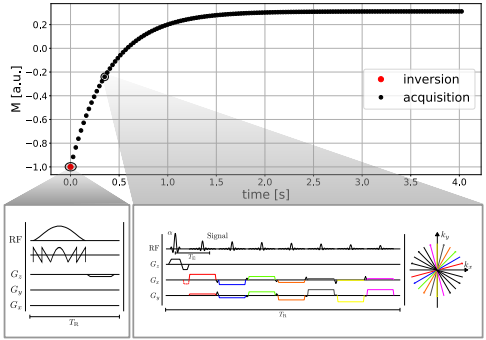
$$\hat{\boldsymbol{x}} = \underset{\boldsymbol{x}}{\operatorname{argmin}} ||P\mathcal{F}CS(\boldsymbol{x}) - Y||_2^2 + R(\boldsymbol{x})$$

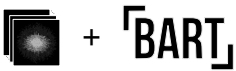
cardiac T1



Wang *et al.*, MRM, 2018.

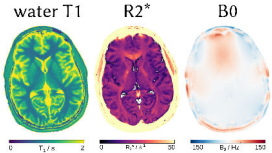
Quantitative (Model-based) MRI Techniques





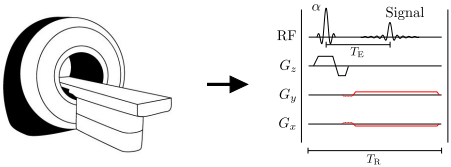
$$S_{\mathrm{TI}_n,\mathrm{TE}_m} = \Big[W_{ss} - (W_{ss}+W_0)\cdot\exp\left(-\mathrm{TI}_n\cdot R^*_{1,W}\right) + (F_{ss}-(F_{ss}+F_0)\cdot\exp\left(-\mathrm{TI}_n\cdot R^*_{1,F}\right)\Big)$$
$$\cdot\exp\left(\mathrm{TE}_m\cdot i2\pi f_{B_0}\right)\cdot\exp\left(-\mathrm{TE}_m\cdot R^*_2\right)$$

$$\hat{\boldsymbol{x}} = \mathrm{argmin}_{\boldsymbol{x}\in D}\sum_{\mathrm{TI}}\sum_{\mathrm{TE}}||\mathcal{PFC}\cdot S_{\mathrm{TI}_n,,\mathrm{TE}_m}(\boldsymbol{x}) - Y_{\mathrm{TI}_n,\mathrm{TE}_m}||_2^2 + R(\boldsymbol{x})$$

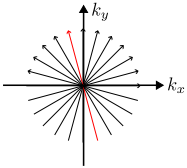


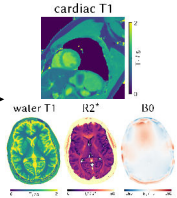
Wang *et al.*, arXiv, 2024.

Motivation



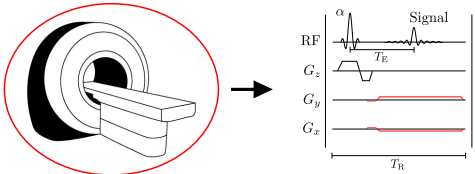
BART





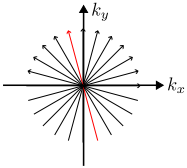
- proprietary sequence implementation
- open-source/reproducible (model-based) reconstruction

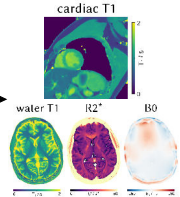
Motivation



=?

BART





- proprietary sequence implementation –> hard to share/reproduce
- open-source/reproducible (model-based) reconstruction –> consistent with sequence?
- reproducibility of qMRI techniques is limited

Outline

- Motivation
- **What do we need to describe a sequence?**
- How can the sequence run on a scanner?
- Is the acquisition and reconstruction consistent?
- How can you use the sequence framework?
- Conclusion

Sequence Framework 1/4: Events and RF Pulse Shapes

- events: start / mid / end time

GRAD

- amplitude
  phase/read/slice - axis

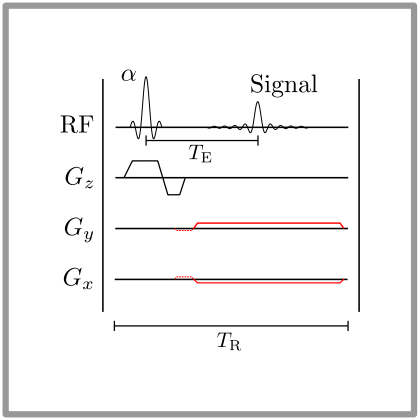
ADC

- frequency / phase
- dwell time

RF

- frequency / phase
- pulse shape
  - samples
  - scaling / integral

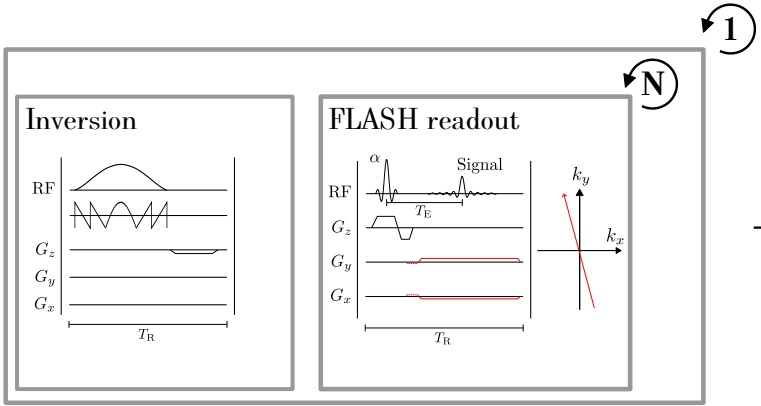
Sequence Framework 2/4: Blocks

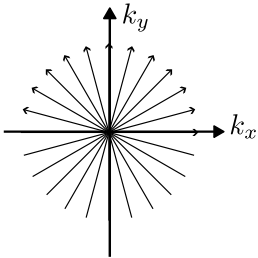


- RF pulse(s)
- ADC(s)
- gradients

- timing

Sequence Framework 3/4: Looping





Sequence Framework 4/4: Parameter Set

- TE / TR / contrast
- Dwell time / base resolution / FOV
- RF pulse shape / duration / flip angle / BWTP
- Encoding pattern
- Imaging position / orientation

- Inversion pulse (type) / TI
- Trigger settings

- System limits (max. gradients, slew rate, etc.)

- Looping order/dimensions (e.g. spokes, slices, frames, inversions, etc.)

Outline

- Motivation
- What do we need to describe a sequence?
- **How can the sequence run on a scanner?**
- Is the acquisition and reconstruction consistent?
- How can you use the sequence framework?
- Conclusion

Sequence Interpretation

BART online

Pulseq export

```
bart seq --BR 256 --lines 377 --raga
        adc gradients moments pul.seq
```

```
config {

    TE/TR = 1.8/3ms
    BR = 256
    FoV = [256,256,6]
    ...
}
```

```
do {
    seq_block(seq_event* ev, config) {
        prepare_rf()
        prepare_gradient()
        prepare_adc()
        ...
    }
} while(seq_continue)
```

GRAD

ADC

RF

BART SEQUENCE

Sequence Interpretation

BART online

Pulseq **export**

```
bart seq --BR 256 --lines 377 --raga
        adc gradients moments pul.seq
```

```
config {

    TE/TR = 1.8/3ms
    BR = 256
    FoV = [256,256,6]
    ...
}
```

```
do {
    seq_block(seq_event* ev, config) {
        prepare_rf()
        prepare_gradient()
        prepare_adc()
        ...
    }
} while(seq_continue)
```

GRAD

ADC

RF

BART SEQUENCE

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377 -f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377 -f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377 -f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

```
# Pulseq sequence file
# Created by BART v1.0.00
# bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377
#        -f 3 --raga adc gradients moments pul.seq

[VERSION]
major 1
minor 4
revision 2

[DEFINITIONS]
AdcRasterTime 1e-07
BlockDurationRaster 1e-05
GradientRasterTime 1e-05
RadiofrequencyRasterTime 1e-06
FOV 0.256 0.256 0.006
PhaseResolution 1.0
TotalDuration 3.97890
```

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377`
  `-f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

```
# Pulseq sequence file
# Created by BART v1.0.00
# bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377
#        -f 3 --raga adc gradients moments pul.seq

[VE [SHAPES]
maj
min shape_id 1                shape_id 2
rev num_samples 620           num_samples 620
    0.0000000000              0.5000000000
[DE 0.0000014131              0.0000000000
Adc 0.0000265792              0.0000000000
Blo 0.0005756503              144
Gra 0.0010369426              -0.5000000000
Rad 0.0092615364              0.0000000000
FOV 0.0097818673              0.0000000000
Pha 0.0144664459              324
Tot 0.0151130175              0.5000000000
    0.0367950723              0.0000000000
    0.0564664459              0.0000000000
    ...                       143
```

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377`
  `-f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

```
# Pulseq sequence file
# Created by BART v1.0.00
# bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377
#        -f 3 --raga adc gradients moments pul.seq

[VE [SHAPES]
maj
min shape_id 1                shape_id 2
rev num_samples 620           num_samples 620
    0.0000000000              0.5000000000
[DE 0.0000014131              0.0000000000
Adc 0.0000265792              0.0000000000
Blo 0.0005756503              144
Gra 0.0010369426              -0.5000000000
Rad 0.0092615364              0.0000000000
FOV 0.0097818673              0.0000000000
Pha 0.0144664459              324
Tot 0.0151130175              0.5000000000
    0.0367950723              0.0000000000
    0.0564664459              0.0000000000
    ...                       143
```

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377 -f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

```
# Format of blocks:
# NUM DUR RF  GX  GY  GZ  ADC  EXT
[BLOCKS]
   1 2040 1 0 0 1 0 0
   2 350 2 0 2 3 1 1


[ADC]
1 512 2000 1887 0.000000 6.134966

[EXTENSIONS]
1 1 1 2
2 1 2 0

#id set labelstring
extension LABELSET 1
1 0 LIN
2 0 REP


[RF]
1 596.058070 4 5 6 800 0.000000 0.000000
2 100.962703 1 2 0 189 0.000000 6.134966


[GRADIENTS]
1 1021813.800000 7 0 0
2 1021813.800000 8 0 0
3 1021813.800000 9 0 0
```

Sequence Interpretation: *Pulseq* Export

- `bart seq --IR_NON --TR 3.5e-3 --BR 256 --lines 377 -f 3 --raga adc gradients moments pul.seq`
- Inversion-recovery random RF spoiled GRE with radial (RAGA) sampling

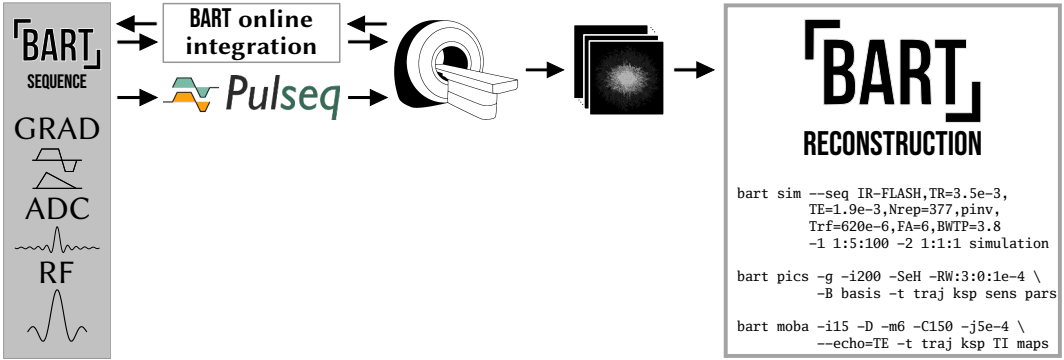
- check on valid parameters (`bart_seq_prepare`)
- write command/definitions to `.seq` - file
- sample all existing RF pulses to `[SHAPES]`
- prepare events of a block (`seq_block`)
- export events to *Pulseq*
  - `[ADC]`: direct conversion including labeling
  - `[RF]`: combine event and pulse shape
  - `[GRADIENTS]`: gradients sampled to arbitrary shape
- repeat for next block (`seq_continue`)

```
# Format of blocks:
# NUM DUR RF  GX  GY  GZ  ADC  EXT
[BLOCKS]
   1 2040 1 0 0 1 0 0
   2 350 2 0 2 3 1 1


[ADC]
1 512 2000 1887 0.000000 6.134966

[EXTENSIONS]
1 1 1 2
2 1 2 0

#id set labelstring
extension LABELSET 1
1 0 LIN
2 0 REP


[RF]
1 596.058070 4 5 6 800 0.000000 0.000000
2 100.962703 1 2 0 189 0.000000 6.134966


[GRADIENTS]
1 1021813.800000 7 0 0
2 1021813.800000 8 0 0
3 1021813.800000 9 0 0
```

Outline

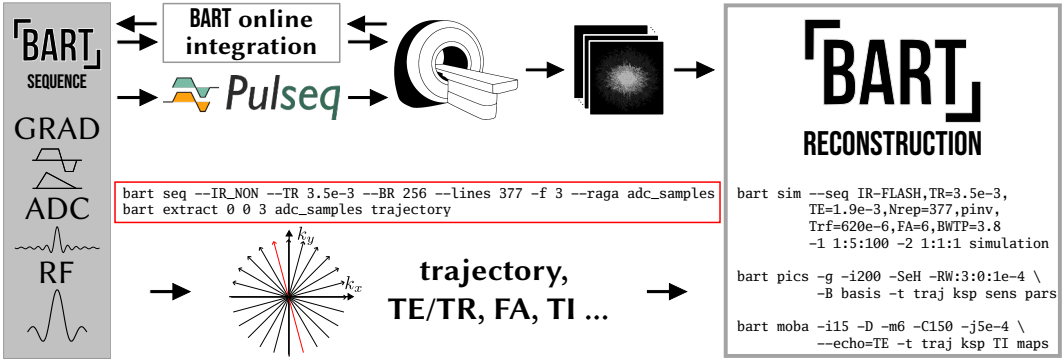
- Motivation
- What do we need to describe a sequence?
- How can the sequence run on a scanner?
- **Is the acquisition and reconstruction consistent?**
- How can you use the sequence framework?
- Conclusion

Consistency



- sequence on scanner via *Pulseq* or BART interpreter
- commandline saved in `.seq` - file or generated from raw-data

Consistency



- sequence on scanner via *Pulseq* or BART interpreter
- commandline saved in `.seq` - file or generated from raw-data
- equal sequence timings and trajectory from same command

Outline

- Motivation
- What do we need to describe a sequence?
- How can the sequence run on a scanner?
- Is the acquisition and reconstruction consistent?
- **How can you use the sequence framework?**
- Conclusion

Application

- BART v1.0.00
  - spoiled GRE with radial (RAGA) sampling
  - support for inversion-recovery, SMS and multi-echo
  - newest version from `https://codeberg.org/mrirecon/bart/`
  - notebook (MRI together 2025)
    `https://gitlab.tugraz.at/ibi/mrirecon/tutorials/bart-workshop/-/tree/master/mri_together_2025`

- *Pulseq* export
  - `bart seq` to write files
  - tested with Siemens *Pulseq* interpreter (v1.5.1)

- `BART online`
  - upcoming: Siemens interpreter sequence (C2P)
  - generate library: `BARTDLL=1 make bart.dll`
  - link interpreter and library
  - change parameters on the fly

Application

- BART v1.0.00
  - spoiled GRE with radial (RAGA) sampling
  - support for inversion-recovery, SMS and multi-echo
  - newest version from `https://codeberg.org/mrirecon/bart/`
  - notebook (MRI together 2025)
    `https://gitlab.tugraz.at/ibi/mrirecon/tutorials/bart-workshop/-/tree/master/mri_together_2025`

- *Pulseq* export
  - `bart seq` to write files
  - tested with Siemens *Pulseq* interpreter (v1.5.1)

- `BART online`
  - upcoming: Siemens interpreter sequence (C2P)
  - generate library: `BARTDLL=1 make bart.dll`
  - link interpreter and library
  - change parameters on the fly

Application

- BART v1.0.00
  - spoiled GRE with radial (RAGA) sampling
  - support for inversion-recovery, SMS and multi-echo
  - newest version from `https://codeberg.org/mrirecon/bart/`
  - notebook (MRI together 2025)
    `https://gitlab.tugraz.at/ibi/mrirecon/tutorials/bart-workshop/-/tree/master/mri_together_2025`

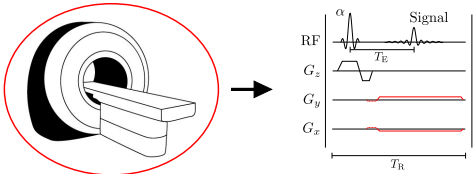
- *Pulseq* export
  - `bart seq` to write files
  - tested with Siemens *Pulseq* interpreter (v1.5.1)

- `BART online`
  - upcoming: Siemens interpreter sequence (C2P)
  - generate library: `BARTDLL=1 make bart.dll`
  - link interpreter and library
  - change parameters on the fly

Outline

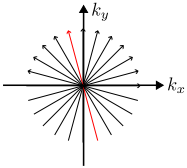
- Motivation
- What do we need to describe a sequence?
- How can the sequence run on a scanner?
- Is the acquisition and reconstruction consistent?
- How can you use the sequence framework?
- **Conclusion**

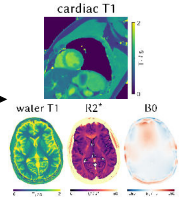
Motivation



=?

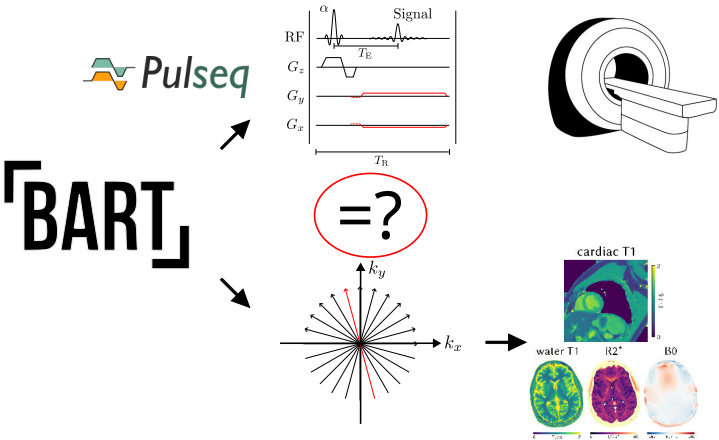
BART





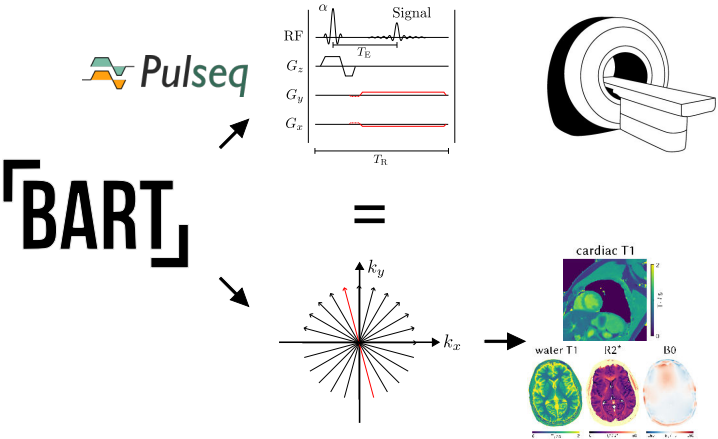
- proprietary sequence implementation –> hard to share/reproduce
- open-source/reproducible (model-based) reconstruction –> consistent with sequence?
- reproducibility of qMRI techniques is limited

Conclusion



- ~~proprietary sequence implementation~~ –> open-source sequence implementation
- open-source/reproducible (model-based) reconstruction –> consistent with sequence?
- reproducibility of qMRI techniques is limited

Conclusion



- ~~proprietary sequence implementation~~ –> open-source sequence implementation
- open-source/reproducible (model-based) reconstruction –> consistent with sequence!
- end-to-end open-source framework for fully replicable qMRI

BART Sequence Framework

Daniel Mackner
Institute of Biomedical Imaging
Graz University of Technology

Vendor-Agnostic Pulse Sequence Programming with Pulseq II

February 24th, 2026